

Report Permessi di Linux

S10L2

Macchina	Ruolo	IP	OS / Servizi
Kali Linux	Sistema principale	-	zsh - utente kali (owner)
-	Utente secondario	-	test_user (others)

1. Obiettivo

L'esercitazione S10L2 ha come obiettivo la configurazione e la verifica dei permessi di lettura, scrittura ed esecuzione in un sistema Linux. Lo scenario scelto prevede la creazione di un file riservato segreto.txt nella directory /home/kali/lab_permessi/, sul quale vengono applicate due diverse configurazioni di permessi tramite il comando chmod, verificando in entrambi i casi il comportamento del sistema rispetto all'utente owner (**kali**) e a un utente secondario senza privilegi (**test_user**).

La scelta del file segreto.txt come oggetto di test è motivata dalla volontà di simulare uno scenario realistico in cui un file contenente dati sensibili deve essere protetto dall'accesso di utenti non autorizzati, applicando il principio del minimo privilegio.

2. Creazione della Directory e del File

Step 1 Creazione directory lab_permessi e file segreto.txt

Viene creata la directory di lavoro lab_permessi nella home dell'utente kali, e al suo interno viene creato il file segreto.txt con contenuto testuale tramite redirectione. Il comando ls -la mostra i permessi assegnati di default dal sistema (umask 022): -rw-r--r-- (644 - owner legge/scrive, tutti gli altri solo lettura).

```
mkdir ~/lab_permessi
touch ~/lab_permessi/segreto.txt
echo "contenuto riservato" > ~/lab_permessi/segreto.txt
ls -la ~/lab_permessi/

# Output permessi iniziali (default umask 022):
-rw-r--r-- 1 kali kali 20 Jun 9 08:51 segreto.txt
```

```
kali@kali ~  
Session Actions Edit View Help  
- (kali@kali)~  
$ mkdir ~/lab_permessi  
- (kali@kali)~  
$ touch ~/lab_permessi/segreto.txt  
- (kali@kali)~  
$  
- (kali@kali)~  
$ echo "contenuto riservato" > ~/lab_permessi/segreto.txt  
- (kali@kali)~  
$ ls -l  
total 138168  
-rw-rw-r-- 1 kali kali 33 Jun 8 2021 allowed.userlist  
-rw-rw-r-- 1 kali kali 62 Apr 20 2021 allowed.userlist.passwd  
drwxr-xr-x 8 kali kali 4096 Jun 3 17:58 Desktop  
drwxr-xr-x 3 kali kali 4096 Jun 3 17:58 Documents  
drwxr-xr-x 2 kali kali 4096 Jun 5 04:36 Downloads  
-rw-rw-r-- 1 root root 2748 May 15 09:58 full_scan.gmap  
-rw-rw-r-- 1 root root 14468 May 15 09:58 full_scan.nmap  
-rw-rw-r-- 1 root root 76406 May 15 09:58 full_scan.xml  
-rwxrwxr-x 1 kali kali 252679 Apr 22 12:10 gameshell-save.sh  
-rw-rw-r-- 1 kali kali 212421 Mar 5 05:43 gameshell.sh  
-rw-rw-r-- 1 kali kali 197 May 14 10:35 hashes_full.txt  
-rw-rw-r-- 1 kali kali 166 May 14 10:28 hashes.txt  
-rw-rw-r-- 1 kali kali 708 May 22 12:59 hash.txt  
drwxrwxr-x 2 kali kali 4096 Jun 9 08:50 lab_permessi  
drwxr-xr-x 2 kali kali 4096 Mar 31 21:42 Music  
drwxr-xr-x 2 kali kali 4096 Mar 31 21:42 Pictures  
-rw-rw-r-- 1 kali kali 30606 May 21 09:11 pmw2pqr.jpeg  
drwxr-xr-x 2 kali kali 4096 May 21 09:33 Projects  
drwxr-xr-x 2 kali kali 4096 Mar 31 21:42 Public  
drwxr-xr-x 2 kali kali 4096 Mar 31 21:42 Templates  
drwx----- 3 kali kali 4096 May 21 12:36 tor-browser  
-rw-rw-r-- 1 kali kali 137732500 May 20 23:49 tor-browser-linux-aarch64-tbb-nightly.2026.05.21.tar.xz  
-rw-rw-r-- 1 kali kali 32 May 14 11:07 username.txt  
-rw-rw-r-- 1 kali kali 31 May 15 10:41 users.txt  
-rw-rw-r-- 1 kali kali 31 Mar 3 2018 users.txt.bk  
drwxr-xr-x 2 kali kali 4096 Mar 31 21:42 Videos
```

3. Modifica dei Permessi con chmod

Step 2 chmod 640 - Permessi differenziati owner/group/others

La prima modifica applica chmod 640: il valore ottale 640 si decompone in **6** (owner: rw-), **4** (group: r--), **0** (others: ---). Questa configurazione garantisce che solo l'owner possa leggere e scrivere il file, gli utenti del gruppo possano solo leggerlo, e tutti gli altri (others) non abbiano alcun accesso. Il file non è eseguibile per nessuno: questa scelta è corretta poiché segreto.txt è un file di dati, non uno script.

```
chmod 640 ~/lab_permessi/segreto.txt  
ls -l ~/lab_permessi/segreto.txt  
  
# Output:  
-rw-r----- 1 kali kali 20 Jun 9 08:51 /home/kali/lab_permessi/segreto.txt
```

```
(kali@kali)~  
$ chmod 640 ~/lab_permessi/segreto.txt  
  
(kali@kali)~  
$ ls -l ~/lab_permessi/segreto.txt  
-rw-r----- 1 kali kali 20 Jun 9 08:51 /home/kali/lab_permessi/segreto.txt
```

Step 3 chmod 444 - Sola lettura universale

La seconda configurazione applica chmod 444: tutti e tre i valori sono 4 (r--), rendendo il file leggibile da chiunque ma non modificabile da nessuno, nemmeno dall'owner. Questo scenario simula un file di log o di configurazione che deve essere consultabile ma non alterabile, ad esempio da processi di audit. Il risultato con ls -l è -r--r--r--.

```
chmod 444 ~/lab_permessi/segreto.txt
ls -l ~/lab_permessi/segreto.txt

# Output:
-r--r--r-- 1 kali kali 30 Jun 9 08:59 /home/kali/lab_permessi/segreto.txt
```

```
(kali㉿kali)-[~/lab_permessi]
$ chmod 444 ~/lab_permessi/segreto.txt

(kali㉿kali)-[~/lab_permessi]
$ ls -l ~/lab_permessi/segreto.txt
-r--r--r-- 1 kali kali 30 Jun 9 08:59 /home/kali/lab_permessi/segreto.txt
```

4. Test dei Permessi con Utente Secondario

Step 4 Tentativo di accesso da test_user (others) - accesso negato

Per verificare il corretto funzionamento della configurazione chmod 640, viene simulato un accesso al file da parte dell'utente **test_user**, che appartiene alla classe others (non è né owner né membro del gruppo kali). Il comando su test_user -c esegue il cat del file utilizzando il path assoluto /home/kali/lab_permessi/segreto.txt. Il sistema risponde con **Permission denied**: test_user rientra nella classe others, che con chmod 640 ha valore 0 (---), confermando che il controllo degli accessi funziona correttamente.

```
su test_user -c "cat /home/kali/lab_permessi/segreto.txt"
Password: ****

# Output:
cat: /home/kali/lab_permessi/segreto.txt: Permission denied
```

```
(kali㉿kali)-[~/lab_permessi]
$ su test_user -c "cat /home/kali/lab_permessi/segreto.txt"
Password:
cat: /home/kali/lab_permessi/segreto.txt: Permission denied
```

5. Riepilogo Configurazioni di Permessi

Configurazione	Notazione	Owner	Group	Others
Default (umask 022)	644 - rw-r--r--	rw- (6)	r-- (4)	r-- (4)
chmod 640	640 - rw-r-----	rw- (6)	r-- (4)	--- (0)
chmod 444	444 - r--r--r--	r-- (4)	r-- (4)	r-- (4)

6. Flusso Completo dell'Esercitazione

#	Fase	Comando	Risultato
1	Creazione	mkdir lab_permessi && touch segreto.txt	File creato, permessi default 644
2	Verifica iniziale	ls -l segreto.txt	Output: -rw-r--r-- (644)
3	chmod 640	chmod 640 segreto.txt && ls -l	Output: -rw-r----- (owner rw, others ---)
4	chmod 444	chmod 444 segreto.txt && ls -l	Output: -r--r--r-- (tutti read-only)
5	Test accesso	su test_user -c 'cat /home/kali/.../segreto.txt'	Permission denied - others: 0 bit ✓

7. Relazione - Motivazione delle Scelte e Analisi dei Risultati

Motivazione delle scelte sui permessi

La scelta di operare su un file denominato `segreto.txt` è intenzionale: simula un documento contenente dati riservati (credenziali, configurazioni sensibili, chiavi) che in un ambiente reale deve essere protetto dall'accesso di utenti non autorizzati.

chmod 640 rappresenta la configurazione più realistica per un file riservato in un sistema multi-utente: l'owner ha pieno controllo in lettura e scrittura, il gruppo (ad esempio un team di amministratori) può consultare il file senza modificarlo, e tutti gli altri utenti sono completamente esclusi. L'assenza del bit di esecuzione è corretta e voluta: un file di testo non deve mai essere eseguibile, poiché ciò costituirebbe un potenziale vettore di attacco (es. esecuzione accidentale di codice malevolo iniettato nel file).

chmod 444 è invece appropriato per file di sola consultazione, come log di sistema o file di configurazione di riferimento: impedisce modifiche anche da parte dell'owner, garantendo l'integrità del contenuto. Questa configurazione è comune nei file di sistema (es. `/etc/passwd`) dove la scrittura deve avvenire solo tramite strumenti dedicati con privilegi elevati.

Analisi dei risultati ottenuti

I test hanno confermato che il sistema di permessi Linux funziona esattamente come atteso. Con `chmod 640` attivo, quando `test_user` ha tentato di leggere il file tramite `cat`, il sistema ha risposto con `Permission denied: test_user` appartiene alla classe `others`, che con questa configurazione ha valore `0` quindi nessun permesso. Il risultato è netto e non lascia margini di ambiguità.

Durante l'esercitazione ho incontrato un problema pratico interessante: il comando su `test_user -c "cat ~/lab_permessi/segreto.txt"` falliva con `No such file or directory`. La causa era che la tilde `~` si espande nella home dell'utente che esegue il comando ed in questo caso `test_user` e non nella home di `kali` dove il file si trovava effettivamente. Risolvere il problema usando il path assoluto `/home/kali/lab_permessi/segreto.txt` ha reso il test corretto e ha anche evidenziato un errore sottile ma comune nella gestione dei permessi in ambienti multi-utente: non dare mai per scontato che `~` punti sempre alla stessa directory.